Report Covering the creation and implementation of the website needed for the CSCU9B3 PHP assignment

3546619

***Abstract*—This document is a report detailing:**
The Use case Architecture Design Database Design Implementation Testing Data Management Considerations of the Event Horizon website (neat name huh ?)

I. Use Case

A. Booking a Technology Workshop

Primary Actor: Registered User

Goal: To reserve a place at an available technology workshop trough the Event Horizon website.

Preconditions:

- The user has an existing account
- The user has access to the Event Horizon website
- At least one workshop is available to book

Trigger: The user chooses an event they wish to attend.

B. Main Success Scenario

1. The user visits the website
2. The user navigates to the login page and enters their credentials
3. The website validates the credentials and grants access
4. The user navigates to the events page where the events details are displayed
5. The user selects and event that has available spaces
6. The user chooses the option to book the selected event
7. The website confirms that the booking has been successful
8. The user can navigate to their bookings page to view their reservation

C. Additional User Outcome

After a successful reservation the user can access the My Bookings page to review their bookings and if they no longer wish to attend they can cancel the reservation, which is then reflected in the booking no longer showing up to that one user and a space becoming available on the Events page.

II. Architecture Design



Fig. 1. The entity relationship diagram used to make the connections between tables

The website uses a three tier architecture consisting of a web browser, and Apache web server running PHP and MySQL relational database. The browser provides the user interface using HTML and CSS, allowing users to log in, browse events, and manage bookings. When a user performs an action the browser sends a HTTP request to the Apache server, where PHP processes the request and communicates with the database using SQL queries. The database retrieves or modifies relevant data and returns the results to PHP, which then generates the appropriate HTML response for the browser. This architecture keeps the UI, application logic and data storage separate.

III. Database design

A. Database Overview

The websites use a MySQL database called event _horizon. It contains three tables; users, EVENTS and bookings. Each table has a specific purpose, allowing the website to store user accounts, the bookings made by users and available events.

B. Entities and Attributes

The users table contains the data for each registered account, this includes the unique user ID, their username and password

hash (please note for the sake of this assignment the password has is the same for all users, in a real website implementation there would be a hash generator tied to the users table along with a sign up option, but that was not a requirement here so i did not add one for the sake of not making this assignment take even longer), along with their email address (also note none of the email address are real and neither are the users, they are all fictional characters, and one is even a fictional planet, again for the sake of simplicity and to not expose any real user data). Passwords are stored as hashes for better security as well, rather than plain text.

The EVENTS table stores details about the workshops, this includes, dates, capacity, description and the events name. This allows the website to dynamically update its capacity and display that information to the users, rather than hard coding it using HTML.

And lastly the bookings table keeps track of per user reservations, containing the users ID, the events ID and the booking date, along with the booking ID and when the booking was created. This allows for each user to only see their own bookings and for there to be no double bookings.

C. Primary and Foreign Keys

Each table has its own primary key, which is used to identify individual records. The user_id primary key is in the users table, event_id is the primary key for the events table and booking_id is the primary key for the bookings table.

Moreover the bookings table also contains 2 foreign keys, thus, linking each booking to the user who made it and the event they selected. This allows a user to make multiple bookings, while allowing an event to have multiple users booking it.

There also is a unique constraint applied to the user _id and event _id to prevent users booking the same event twice. Foreign key constraints also ensure the bookings remain linked to valid users and events, with associated bookings being deleted if the referenced records are removed.

D. How the Database Supports the Website

The database provides the info needed for the features of our website. When a user visits the events page, the website pulls the data from the database and calculates the remaining space for each one.

When a user attempts to make a booking, the system checks that wether the event is still able to accept more bookings and that the users hasn't already booked that event. If these conditions are met then the new booking is stored and tied to the user and the database updates the available spaces.

Users can also view their current bookings and cancel them if they wish when said booking is canceled its record is removed and the website recalculates available spaces to show other users that there is 1 extra free slot.

IV. Implementation

A. Overview

The website was developed using 4 coding languages, those being HTML, CSS, PHP and MySQL. HTML gave the website its main body and structure while CSS gave its colours and overall UI look. PHP handles the main functionality and communication to our database, along with MySQL also being the basis of our database.

B. User Authentication

Users can login using an existing username and password. PHP communicates with the database to retrieve the relevant account info and checks wether the input password mathces the stored password hash using password _verify(), and if it finds the two match, and new session is created in order to keep the user logged in while navigating. A invalid set of credentials results in an error message instead.

C. Event Listings and Booking

The events page actively retrieves event information, such as dates, description and available spaces. We also actively calculate the remaining spaces by subtracting all bookings made by active users.

Bookings are also checked using PHP, to ensure that an event has the capacity for that user, that the user hasn't already booked the event, and that the event exists, before saving anything into the database, once all these checks are passed, a new record is inserted into the bookings table.

D. Managing Bookings

Users can access the My Bookings page in order to view their existing reservations. Each bookings is linked to the currently logged in account, ensuring users only see their own reservations.

Users can also cancel bookings. The user can cancel their own bookings, which in turn updates the database to reflect the change in available spaces.

E. HTML, PHP and SQL Interaction

The website combines HTML, PHP and SQL to provide its overall functionality. HTML displays the pages and forms, PHP processes user input and application logic, along with communication between the browser and the database, while SQL retrieves and modifies the stored information.

For example, the following SQL query retrieves event information and calculates the remaining capacity:

```
SELECT
    e.event_name
    e.capacity - (
        SELECT COUNT(*)
        FROM bookings b
        WHERE
        b.event_id = e.event_id

    ) AS places_left
FROM EVENTS e;
```

PHP then executes the query using PDO (PHP Data Objects) and displays the returned information on the events page. Prepared statements are also used when processing user input to prevent SQL injection attacks.

V. Testing

A. Test 1:

Testing to see wether navigation stays consistent between pages with the user logged in. Figure 2 shows the events page of the website only displaying 3 navigation options while a user is logged in, it should display 5, figures 3 and 4 show the extra pages that should be seen in the navigation bar of the website.

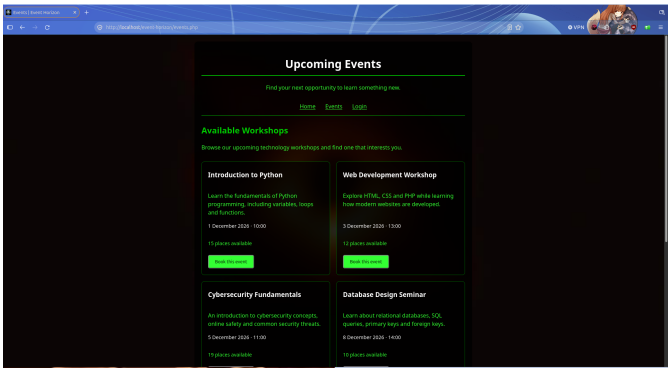


Fig. 2. Showing wrong navigation

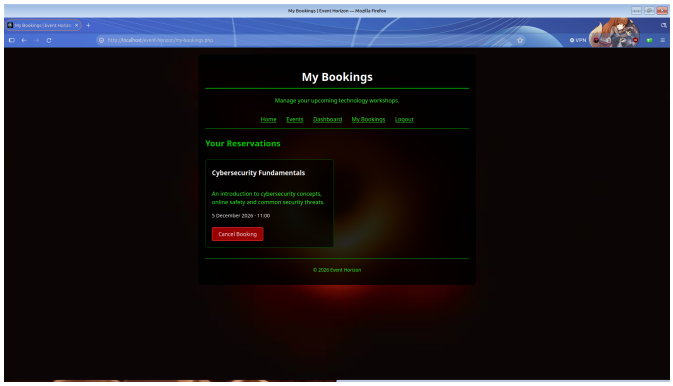


Fig. 3.

This was fixed after realizing that each page had a hard coded nav block, and was fixed by making another php filed called header.php which holds all the navigation locations and has been used to replace all the navigation blocks of all relevant code and is shown to be working in figure 4.

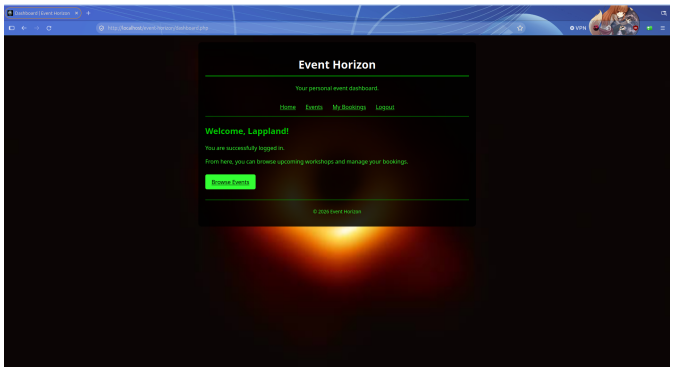


Fig. 4. Showing working website

B. Test 2:

We are testing to see how the website reacts to valid and invalid credentials.

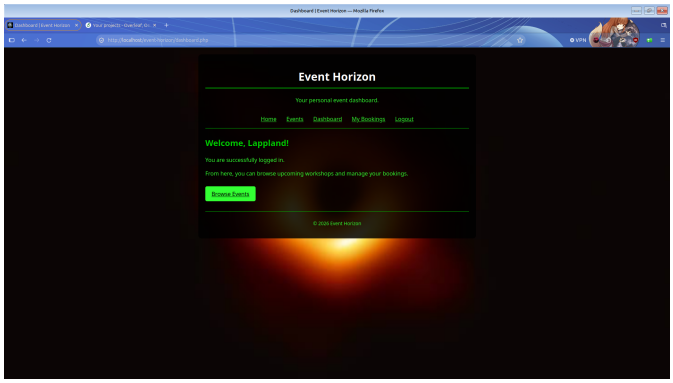


Fig. 5. Shows valid credentials, and is working as expected.

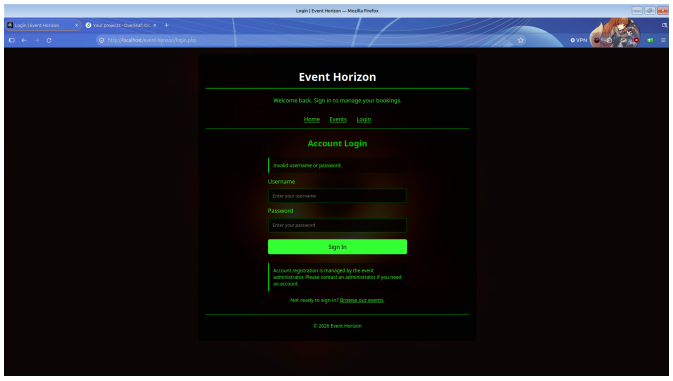


Fig. 6. Showing invalid credentials and is also working as planned.

C. Test 3:

Here we test full website navigation while logged in and logged out. A logged in user should see 5 navigation options (Home, Events, Dashboard, My Bookings and Logout) while a logged out user should only see 3 options (Home, Events, Login).

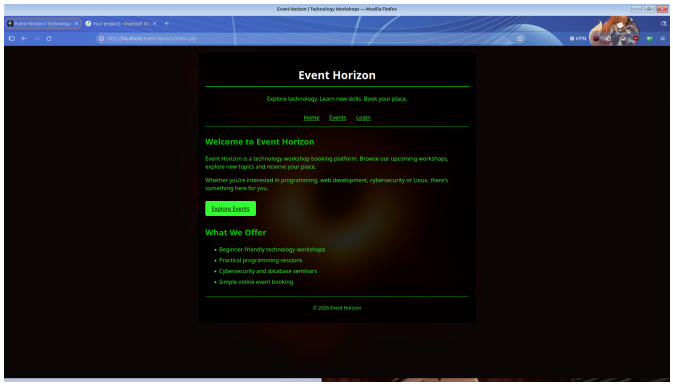


Fig. 7. Shows logged in user navigation working as expected.

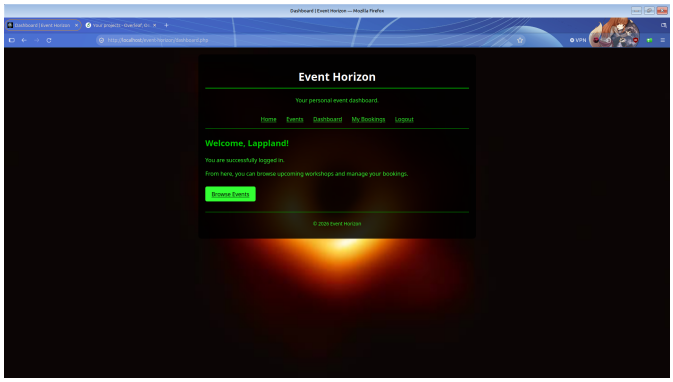


Fig. 8. Shows a logged out user, also working as expected.

D. Test 4:

Here we test the booking functionality, here we should see a logged in user be able to book an event and that even to show up in the My Bookings section.

Figure 9 and 10 show the user booking the event and the event showing up in the expected section.

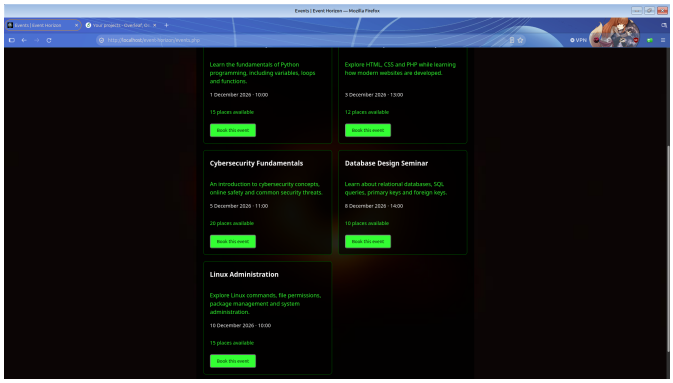


Fig. 9. User books a session.

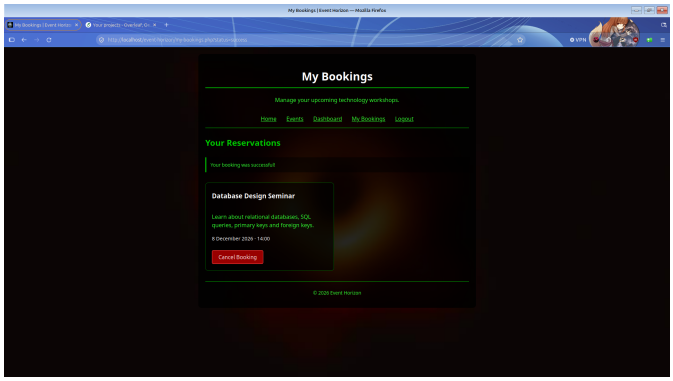


Fig. 10. Booked session shows up in the My Bookings page.

E. Test 5:

In this test we will test a logged in user canceling a booking. The user should be able to simply press cancel booking and the booking should no longer be on their My Bookings page.

Figures 11 and 12 show the booking system working as expected

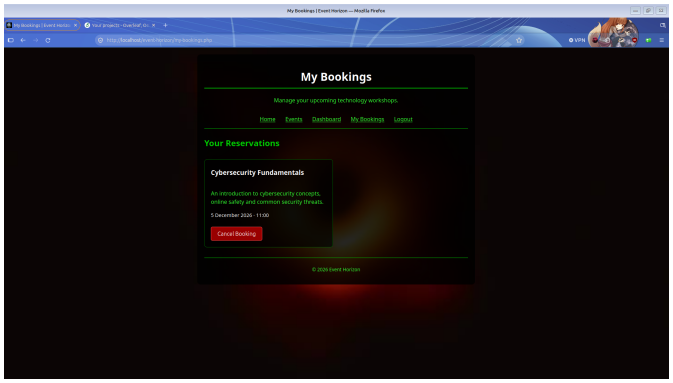


Fig. 11. Booked session is canceled by user.

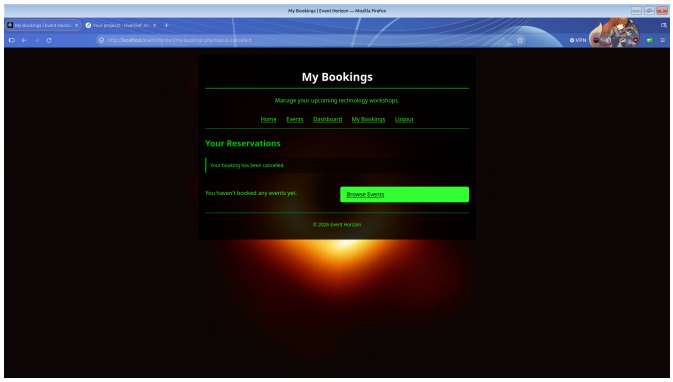


Fig. 12. User successfully cancels their booking.

F. Test 6:

Here we log in as another user and check to see wether the website exposes another users bookings.

Figure 14 shows the other user being unable to see the other users booking. (please note i did not do a full pent test and will not be doing a pen test for this assignment, but as far as i programmed this website it should not be vulnerable to SQL injections or other types of attack that target the database tied to this website)

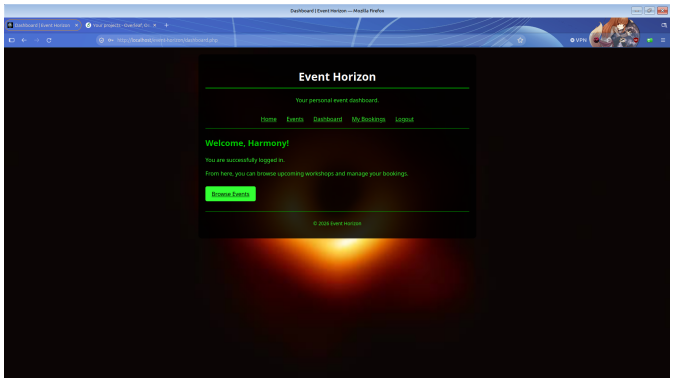


Fig. 13. Here we log in as another user.

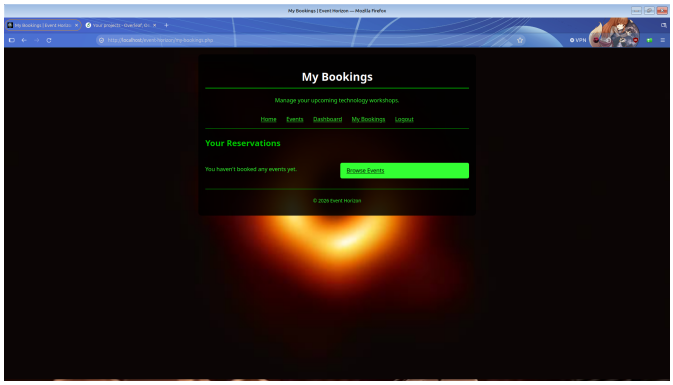


Fig. 14. Here we see that another user will not see any other users bookings.

G. Test 7:

In this test we will see whether the website allows a user to book the same event twice, expected behavior should not let the user book the same event twice

Figure 15 shows that a user cannot book the same event twice.

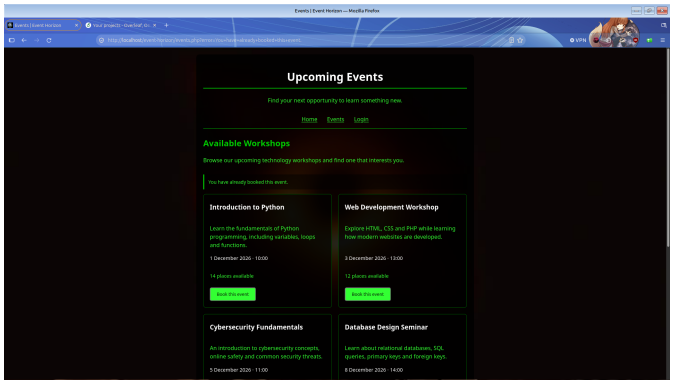


Fig. 15. Here we see a user unable to book the same event twice.

VI. Data Management Considerations

If Event Horizon were deployed for public use, it would collect and store personal information such as usernames (which some users can input as their real names rather than a internet alias), email addresses (which is personally identifiable data), and booking records. This information needs to be handled responsibly to protect users privacy and prevent unauthorised access.

From a technical perspective, passwords should never be stored as plain text, althoug even a password hash can be cracked given time, but it is extremely hard and nearly impossible to crack a password hash, hence why our website uses them. Moreover predefined SQL statements reduce the risk of SQL injections, while also having session based authentication to prevent unwanted elevated access. Database access should also be limited to authorised accounts only, and implement user access rights to avoid users having access to things they should not. Also regular backups will benefit the long term running and maintanace of the website as a whole.

There are also legal aspects to consider. As the website would handle personal infomration, it needs to comply with the UK General Data Protection Regulation (UK GDPR) and the Data Protection Act 2018. These two acts make companies holding data legally responsible for the care and storage of user data, and it makes companies inform users of what information is collected, for how long its stored, how its used, and their right to removal (usually covered in a End User License Agreement or a User Agreement).

Moreover, ethically speaking, users should be able to trust that their information will only be used for reasons outlined in the EULA and that none of their data would be used for any other reason or sold unnecessarily, or that data is not kept secure.

Ensuring compliance with data protection laws, good security practice and prevention of attacks, and being transparent with data use will ensure user trust, legal standing and overall functionality of the Event Horizon website.